

Curso <i>Engenharia de Software</i>	Campus <i>Rio Verde</i>
<i>Programação Orientada a Objetos</i>	
Nome do(a) acadêmico(a)	Valor 7,0
Professor <i>Sergio Souza Novak</i>	Turma <i>B</i>
Data da Avaliação 20/03/2026	Código <i>ESW448</i>

ATENÇÃO: Somente serão passíveis de **REVISÃO** avaliações resolvidas à **TINTA**.

ORIENTAÇÕES PARA A RESOLUÇÃO

- A avaliação é individual, sem consulta;
- A interpretação de texto faz parte da avaliação;

QUESTÕES

- (ENADE 2017):** O encapsulamento é um mecanismo da programação orientada a objetos no qual os membros de uma classe (atributos e métodos) constituem uma caixa preta. O nível de visibilidade dos membros pode ser definido pelos modificadores de visibilidade "privado", "público" e "protegido". Com relação ao comportamento gerado pelos modificadores de visibilidade, assinale a opção correta.
 - Um atributo privado pode ser acessado pelos métodos privados da própria classe e pelos métodos protegidos das suas classes descendentes.
 - Um atributo privado pode ser acessado pelos métodos públicos da própria classe e pelos métodos públicos das suas classes descendentes.
 - Um membro público é visível na classe à qual ele pertence, mas não é visível nas suas classes descendentes.
 - Um método protegido não pode acessar os atributos privados e declarados na própria classe.
 - Um membro protegido é visível na classe à qual pertence e em suas classes descendentes.
- Encapsulamento:** Observe o diagrama da classe **ContaBancaria** abaixo. Assinale a alternativa que apresenta a implementação correta em Java, respeitando o encapsulamento e a lógica dos métodos.

ContaBancaria
- saldo : double + depositar(valor: double) : void + sacar(valor: double) : void + getSaldo() : double

- A)

```
public class ContaBancaria {
    public double saldo;
    public void depositar(double v) { saldo = v; }
}
```
- ```
public class ContaBancaria {
 private double saldo;
 public ContaBancaria(double s) { this.saldo = s; }
 public void depositar(double v) { this.saldo += v; }
 public void sacar(double v) { this.saldo -= v; }
 public double getSaldo() { return this.saldo; }
}
```
- B) 

```
public class ContaBancaria {
 private double saldo;
 public void depositar(double v) { }
 public void sacar(double v) { }
}
```
- ```
public class ContaBancaria {
    private double saldo;
    private double getSaldo() { return saldo; }
}
```
- B)

```
public class ContaBancaria {
    public double getSaldo() { return 100.0; }
}
```

3. Na programação orientada a objetos, os modificadores de acesso controlam a visibilidade de classes, atributos e métodos. Assinale a alternativa que apresenta o modificador que permite que um membro seja acessado apenas dentro da própria classe e por suas subclasses:

- A) public
- B) private
- C) protected
- D) package-private
- E) static

4. **Construtores:** Qual é a principal finalidade de um método construtor em uma classe?

- A) Destruir o objeto quando ele não for mais necessário.
- B) Inicializar os atributos de um objeto no momento de sua criação.
- C) Permitir que o objeto seja herdado por outras classes.
- D) Converter um objeto em uma string para exibição.
- E) Impedir que o objeto seja clonado.

5. **Indústria 4.0 — Máquina CNC:** No contexto de sistemas de manufatura inteligentes, a classe MaquinaCNC foi projetada para representar uma máquina de usinagem com controle automatizado.

MaquinaCNC

```
- modelo : String
- horasOperacao : double
- temperaturaAtual : double
- temperaturaMaxima : double
- ligada : boolean
- pecasProduzidas : int
+ MaquinaCNC(modelo: String, tempMax: double)
+ ligar() : void
+ desligar() : void
+ produzir(qtd:int) : boolean
+ getTemperaturaAtual() : double
+ precisaManutencao() : boolean
- controlarTemperatura(qtd:int) : void
- verificarManutencao() : boolean
```

```
01: // MaquinaCNC.java
02: public class MaquinaCNC {
03:     private String modelo;
04:     private double horasOperacao;
05:     private double temperaturaAtual;
06:     private double temperaturaMaxima;
07:     private boolean ligada;
08:     private int pecasProduzidas;
09:
10:     public MaquinaCNC(String modelo, double tempMax) {
11:         this.modelo = modelo;
12:         this.temperaturaMaxima = tempMax;
13:         this.temperaturaAtual = 25.0; // temperatura ambiente
14:     }
15:
16:     public void ligar() { this.ligada = true; }
17:     public void desligar() { this.ligada = false; }
18:
19:     public boolean produzir(int qtd) {
20:         if (!this.ligada){ return false; }
21:
22:         this.pecasProduzidas += qtd;
23:         this.horasOperacao += qtd * 0.1;
24:         this.controlarTemperatura(qtd);
25:         return true;
26:     }
27:
28:     public double getTemperaturaAtual() { return this.
temperaturaAtual; }
29:     public boolean precisaManutencao() { return this.
verificarManutencao(); }
30:
31:     private void controlarTemperatura(int qtd) {
32:         this.temperaturaAtual += qtd * 2.0;
33:         if (this.temperaturaAtual > temperaturaMaxima) {
34:             desligar();
35:         }
```

```

36:     }
37:
38:     private boolean verificarManutencao() {
39:         return horasOperacao > 1000;
40:     }
41: }
42:
43: // Main.java
44: public class Main {
45:     public static void main(String[] args) {
46:         MaquinaCNC cnc = new MaquinaCNC("CNC-X1", 100.0);
47:         cnc.ligar();
48:         if (cnc.produzir(50)) {
49:             System.out.println("Producao concluida.");
50:         }
51:     }
52: }

```

Com base no código da classe MaquinaCNC, analise as afirmações abaixo como Verdadeiras (V) ou Falsas (F) e assinale a alternativa que apresenta a sequência **correta**:

- () Se a linha 14 for removida, a máquina poderá produzir peças (pecasProduzidas) mesmo estando desligada (ligada = false).
- () Os atributos das linhas 03 a 08 estão corretamente encapsulados como private, impedindo o acesso direto de outras classes.
- () O método precisaManutencao (linha 22) expõe diretamente o atributo horasOperacao, violando o princípio do encapsulamento.

- A) V - V - V - F.
- B) V - F - V - V.
- C) F - V - F - V.
- D) V - V - F - F.
- E) F - F - V - V.

6. **Siderurgia — Alto Forno:** A modelagem da classe AltoForno envolve o controle de um equipamento utilizado para produção de metal.

AltoForno
- temperaturaInterna : double - temperaturaMinimaOperacao : double - temperaturaMaximaOperacao : double - volumeMetalProduzido : double - ativo : boolean + AltoForno(tempMin: double, tempMax: double) + iniciarForno() : void + desligarForno() : void + injetarCombustivel(qtd:double) : double + produzirMetal() : double + estaAtivo() : boolean - verificarFaixaTermica() : boolean - controleSeguranca() : void

```

01: // AltoForno.java
02: public class AltoForno {
03:     private double temperaturaInterna;
04:     private double temperaturaMinimaOperacao;
05:     private double temperaturaMaximaOperacao;
06:     private double volumeMetalProduzido;
07:     private boolean ativo;
08:
09:     public AltoForno(double tempMin, double tempMax) {
10:         this.temperaturaMinimaOperacao = tempMin;
11:         this.temperaturaMaximaOperacao = tempMax;
12:         this.temperaturaInterna = 20.0;
13:     }
14:
15:     public void iniciarForno() { this.ativo = true; }
16:     public void desligarForno() { this.ativo = false; }
17:     public boolean estaAtivo() { return ativo; }
18:
19:     public double injetarCombustivel(double qtd) {
20:         this.temperaturaInterna += qtd * 10;
21:         controleSeguranca();
22:         return temperaturaInterna;
23:     }
24:
25:     public double produzirMetal() {
26:         if (ativo && verificarFaixaTermica()) {
27:             double producao = 100.0;
28:             this.volumeMetalProduzido += producao;
29:             return producao;
30:         }
31:         return 0.0;
32:     }
33:
34:     private boolean verificarFaixaTermica() {
35:         return temperaturaInterna >= temperaturaMinimaOperacao
36:             &&
37:             temperaturaInterna <= temperaturaMaximaOperacao
38:             ;
39:     }
40:
41:     private void controleSeguranca() {
42:         if (temperaturaInterna > temperaturaMaximaOperacao) {
43:             desligarForno();
44:         }
45:     }

```

Considerando o código da classe AltoForno e os princípios de Programação Orientada a Objetos, julgue os itens a seguir como Verdadeiras (V) ou Falsas (F) e assinale a sequência **correta**:

- () O método verificarFaixaTermica (linhas 28-31) é private, o que significa que ele só pode ser invocado por outros métodos dentro de outras classes.
- () Na linha 15, o método controleSeguranca é chamado para garantir que a temperatura não ultrapasse o limite após a injeção de combustível, demonstrando o controle de estado interno.

- () A lógica de produção de metal na linha 20 depende exclusivamente do forno estar ativo, ignorando a temperatura atual do equipamento.
- () O atributo `temperaturaInterna` (linha 03) poderia ser `public` sem afetar a segurança do equipamento, pois o método `controleSeguranca` corrigiria qualquer valor inválido.

- A) V - V - F - F.
- B) V - F - V - V.
- C) F - V - V - F.
- D) V - V - V - F.
- E) F - F - F - V.

7. (VUNESP 2021 - Sema de Piracicaba - SP - Programador Junior) Em programação orientada a objetos, métodos de acesso do tipo `setter` têm a finalidade primária de:

- A) obter o valor de um atributo.
- B) modificar o valor de um atributo.
- C) inicializar os valores de atributos de um objeto assim que ele é instanciado.
- D) contar quantas vezes o valor de um atributo foi obtido.
- E) salvar o valor de um atributo de um objeto em um banco de dados.

8. (FUNDATEC 2025 - GHC-RS - Programador) Considerando os conceitos de orientação a objetos na linguagem de programação Java, assinale a alternativa que apresenta quem pode acessar o atributo `v` diretamente:

```
package conceicao;  
public class Conceicao {  
    protected String v;  
}
```

- A) Apenas as classes que estejam no mesmo pacote.
- B) Apenas os métodos da própria classe.
- C) As classes filhas de qualquer pacote.
- D) As classes filhas apenas do mesmo pacote.
- E) Por qualquer classe de qualquer pacote.
9. Considere a modelagem de uma classe `Carro` que possui um atributo privado chamado `velocidadeAtual`. Sobre a importância de se utilizar métodos *getters* e *setters*, assinale a alternativa que descreve corretamente uma vantagem e um exemplo de regra de negócio que poderia ser implementada:

Carro
- <code>velocidadeAtual</code> : <code>double</code>
+ <code>getVelocidade()</code> : <code>double</code>
+ <code>setVelocidade(v: double)</code> : <code>void</code>

- A) A vantagem é permitir que qualquer pessoa altere a velocidade sem restrições. Exemplo: Deixar a velocidade ser negativa.
- B) A vantagem é o encapsulamento, permitindo validar os dados antes de alterar o atributo. Exemplo: Impedir que a velocidade receba valores negativos.

- C) A vantagem é diminuir o tamanho do código Java. Exemplo: Somar a velocidade com o ano do carro.
- D) A vantagem é tornar todos os atributos públicos automaticamente. Exemplo: Alterar a cor do carro no método da velocidade.
- E) A vantagem é que o código roda mais rápido. Exemplo: Multiplicar a velocidade por zero para parar o carro.